Using the interfaces from SLF4J (www.slf4j.org), you have the freedom to change the logging library without changing your Java code. Furthermore, cloud native applications should treat logs as events and stream them to the standard output. That's precisely what Spring Boot does out of the box. Convenient, right?

CONFIGURING LOGGING IN SPRING BOOT

Event logs are categorized by level with decreasing details and increasing importance: trace, debug, info, warn, error. By default, Spring Boot logs everything from the *info* level up.

A *logger* is a class that produces log events. You can set logger levels through configuration properties, with options to apply global configurations or to target specific packages or classes. For example, in chapter 9 we set a *debug* logger to get more details about the circuit breakers implemented with Resilience4J (in the application.yml file in the Edge Service project):

```
logging:
  level:
    io.github.resilience4j: debug     ❶
```

❶ Sets a debug logger for the Resilience4J library

You might need to configure multiple loggers at the same time. In that case, you can collect them in a *log group* and apply the configuration to the group directly. Spring Boot provides two predefined log groups, `web` and `sql`, but you can also define your own. For example, to better analyze the behavior of the circuit breakers defined in the Edge Service application, you could define a log group and configure a log level for both Resilience4J and Spring Cloud Circuit Breaker.